DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Nguyen, Duc Trung, B.Sc.

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Entwurf und Evaluation multiagentenbasierter KI-Systeme für Contract-Lifecycle-Management-SaaS-Plattformen

Author:	Nguyen, Duc Trung, B.Sc.
Supervisor:	Prof. Dr. Ingo Weber
Advisor:	Samed Bayer, M.Sc.
Submission Date:	04.05.2026

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 04.05.2026 Nguyen, Duc Trung, B.Sc.

Acknowledgments

Abstract

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**
1.1 Motivation 1
1.2 Research Gap 2
1.3 Research Questions 3
1.4 Hypotheses 4
1.5 Contributions 4
1.6 Thesis Outline 6

2 Literature Review **7**
2.1 The CLM Problem Space 7
2.2 From Rule-Based to Neural Approaches 8
2.3 LLMs in Contract Analysis 9
2.4 Multi-Agent Systems for Legal AI 11
2.5 When Multi-Agent Helps and When It Does Not 12
2.6 Evaluation Methodology in Legal NLP 14
2.7 Synthesis and Research Positioning 15

3 Methodology **17**
3.1 Research Design 17
3.2 Dataset Selection and Justification 18
3.3 Sampling Strategy: Natural Class Distribution 19
3.4 Metric Selection 19
3.5 Hypotheses and Success Criteria 20
3.6 Statistical Methodology 21
3.7 Model Selection 22
3.8 Threats to Validity 22

4 System Design and Implementation **24**
4.1 Architectural Patterns Evaluated 24

4.2 Baseline Architectures 24
4.3 Multi-Agent Architectures 25
4.4 Implementation Stack 26
4.5 Traceability and Auditability 27
4.6 Experimental Infrastructure 28

5 Results **30**
5.1 Overview of Experimental Runs 30
5.2 H1 — Multi-Agent vs. Single-Agent Baselines 31
5.3 H2 — Rare-Category Gap 33
5.4 H3 — Architecture vs. Prompts 33
5.5 Additional Findings 34
5.5.1 The Near-Perfect Routing Accuracy Critique 34
5.5.2 The Classification-Gate Rejection Cascade 34
5.5.3 Cost-Efficiency Analysis 36

6 Discussion **37**
6.1 When Multi-Agent Helps: Evidence for Pipeline Decomposition 37
6.2 When Multi-Agent Hurts: The LLM-Router Problem 37
6.3 Architecture vs Prompts: The H3 Verdict 37
6.4 Practitioner Perspective: Gaps Between Benchmarks and Real-World CLM 37
6.5 Connecting to Prior Work 37
6.6 Limitations 37

7 Conclusion and Outlook **38**
7.1 Summary of Contributions 38
7.2 Answers to Research Questions 38
7.3 Outlook and Future Work 38

List of Figures **39**

List of Tables **40**

Bibliography **41**

1 Introduction

1.1 Motivation

Contract Lifecycle Management (CLM) is the connective tissue of modern enterprise commerce. Every commercial relationship, employment arrangement, vendor engagement, and regulatory commitment is encoded in a contract, and the operational performance of a firm depends on its ability to draft, review, execute, monitor, and renew these documents at scale. Industry surveys conducted by World Commerce & Contracting indicate that organisations lose value equivalent to roughly nine percent of annual revenue as a consequence of poor contracting practices, with the figure rising to fifteen percent or more in industries with high contractual complexity [Wor25]. The same survey reports that contract-related data is, on average, dispersed across twenty-four distinct systems, and that eighty-three percent of executives consider their contracts too rigid to adapt to changing business conditions [Wor25]. These figures position CLM not as a clerical concern but as a material driver of enterprise financial performance.

The cost structure of manual contract review compounds this strategic exposure. Legal professionals spend a substantial proportion of their billable time on tasks that are, in principle, automatable: locating clauses, comparing them against playbooks, flagging deviations, and tracking obligations. Goldman Sachs research estimates that approximately forty-four percent of legal-task time in the United States is technically exposed to automation by generative artificial intelligence, the second-highest exposure rating across surveyed professions [Hat+23]. Vendor-led empirical comparisons such as the Onit-funded study by Martin et al. report that frontier large language models match or exceed junior lawyers and legal-process outsourcers on issue-identification F-scores while reducing per-document review time by roughly two orders of magnitude [Mar+24]. Although such vendor evidence must be read with appropriate skepticism, the directional finding — that LLMs are competitive on standardised review tasks — is consistent across independent academic benchmarks [Hen+21; Liu+25].

The appeal of LLM automation, however, is not the same as the demonstration of its reliability under enterprise conditions. State-of-the-art models still hallucinate clauses that are not present, omit clauses that are, exhibit pronounced sensitivity to prompt phrasing [Cha+24], and fail in ways that are difficult to audit. Liu et al. document

a *laziness* failure mode in which open-source models default to "no relevant clause" responses even when the relevant text is plainly present in the contract [Liu+25]. Single-prompt zero-shot deployments, which dominate current commercial CLM offerings, give product teams little ability to localise these failures, attribute them to specific model behaviours, or remediate them without retraining or wholesale model substitution.

In response, both the research community and the agentic-AI vendor ecosystem have advocated multi-agent architectures as the next paradigm for legal document automation. The argument is that decomposing contract analysis into specialised cooperating agents — a classifier, an extractor, a reviewer, a router, a verifier — ought to improve accuracy through specialisation, enable explainability through inspectable inter-agent traces, and accommodate enterprise governance requirements such as those imposed by Article 13 of the European Union's Artificial Intelligence Act [EC24] and Article 22 of the General Data Protection Regulation [EC16]. Multi-agent frameworks such as AutoGen [Wu+23], MetaGPT [Hon+24], CAMEL [Li+23], and Mixture-of-Agents [Wan+24] have made these architectures cheap to construct, and orchestration libraries such as LangGraph [Lan24a] have lowered the engineering threshold further. Whether these architectures actually *work* for contract analysis at production scale is, however, an open empirical question — and it is the question this thesis takes up.

1.2 Research Gap

Three observations motivate the research gap that this thesis addresses.

First, the empirical evidence base for which multi-agent patterns improve contract analysis is thin. Recent legal multi-agent systems such as ChatLaw [Cui+24], PAKTON [Rap+25], and L-MARS [WY25] are valuable system-design contributions, but each evaluates a single architecture on a single benchmark, typically without ablating the architectural choice itself. There is, to our knowledge, no published cross-architecture, cross-model-family comparison on a public contract-analysis benchmark that isolates the contribution of the multi-agent design from the contribution of the underlying LLM.

Second, recent work on multi-agent systems outside the legal domain has begun to question the universality of multi-agent gains. Gao et al. argue that the benefits of multi-agent systems over single-agent systems diminish as base-model capability increases, because frontier models with stronger long-context reasoning, memory, and tool-use absorb the workload that previously required decomposition [Gao+25]. Cemri et al. construct a taxonomy of fourteen failure modes specific to multi-agent LLM systems, grouped into specification, inter-agent misalignment, and verification-and-termination categories, and report that simple interventions yield only modest gains [Cem+25].

Bogavelli et al., in their AgentArch benchmark, find that no agent architecture is universally optimal across model families and that smaller models can outperform larger models when matched to an appropriate architecture [Bog+25]. These results suggest that whether multi-agent helps in CLM depends on conditions that have not yet been mapped.

Third, prior contract-analysis evaluations have used sampling and metric choices that may obscure the dominant failure mode. The CUAD benchmark exhibits a heavily imbalanced class distribution at the contract-clause level, but several published evaluations resample to a positive-heavy ratio, which produces optimistic precision estimates and hides over-extraction failures [Liu+25]. Sampling design and metric weighting interact in ways that change the ranking of architectures, and these interactions have not been systematically interrogated.

Taken together, these observations indicate a gap that is simultaneously empirical, mechanistic, and methodological: the field lacks a rigorous comparison of multi-agent architectural patterns for clause-level contract analysis under realistic class distributions, with statistical hypothesis tests, mechanistic diagnosis of failure modes, and explicit treatment of when multi-agent designs help versus when they do not.

1.3 Research Questions

This thesis investigates the following main research question:

> *Under what conditions do multi-agent architectures improve contract analysis accuracy, efficiency, and explainability in enterprise CLM systems, and which architectural patterns are most effective?*

Three sub-questions decompose this main question into empirically tractable parts.

Sub-RQ1. Which multi-agent architectural patterns — pipeline decomposition, specialist routing, or consolidated prompting — are most effective for clause-level contract analysis?

Sub-RQ2. How do multi-agent approaches compare to single-agent baselines in clause extraction accuracy and cost-efficiency across diverse clause categories and contract lengths?

Sub-RQ3. What architectural decisions enable auditable reasoning traces in multi-agent contract analysis, and what design tradeoffs do they introduce?

The first sub-question targets architectural ranking conditional on model family. The second sub-question incorporates economic and class-distribution considerations that govern enterprise deployability. The third sub-question treats explainability as

a system-design property rather than as a metric to be optimised, in line with the regulatory framing imposed by Articles 13 and 86 of the EU AI Act [EC24].

1.4 Hypotheses

The empirical core of the thesis tests three hypotheses. A fourth hypothesis on traceability, present in the original thesis proposal, has been reframed: rather than test whether multi-agent designs produce more inspectable traces — a property that is largely determined by the orchestration layer rather than the architecture per se — we treat traceability as a system property of the implementation and document the design choices that secure it (see Sub-RQ3).

H1. *At least one multi-agent architecture outperforms the strongest single-agent baseline on F_2.* The rationale is that decomposition should reduce per-step prompt length and isolate the classification decision from the extraction decision, which has been argued to improve recall in imbalanced retrieval tasks [Liu+25]. We test H1 with a paired McNemar's test [McN47] between the best multi-agent configuration and the best single-agent configuration per model, with effect sizes reported.

H2. *Multi-agent architectures reduce the rare-common F_2 performance gap compared to single-agent baselines.* CUAD's clause categories vary substantially in prevalence, and architectures that route to category-specialised prompts may benefit rare categories disproportionately. We test H2 by comparing the rare-versus-common F_2 gap across architectures using a paired bootstrap confidence interval on the gap difference.

H3. *Specialist architecture differs significantly from consolidated specialist prompts.* If routing-plus-specialists outperforms a single prompt that contains all specialist guidance, the difference must be attributable to the routing or to the per-specialist context isolation. We test H3 with a two-sided McNemar's test contrasting the LLM-routed multi-agent configuration against a single-agent ablation that contains the union of specialist instructions, and additionally contrast the pipeline-decomposition configuration against the consolidated prompt.

The reader is alerted that the empirical results, reported in Chapter 5, are mixed with respect to these hypotheses, and that the thesis treats the negative or null findings as substantive contributions rather than as failures of the design.

1.5 Contributions

This thesis advances five contributions, which we summarise here and develop in subsequent chapters.

The first contribution is methodological. We introduce a natural class-distribution sampling correction for CUAD-based evaluation, in which the positive-to-negative ratio of clause-presence labels is held at approximately thirty to seventy, matching CUAD's underlying distribution rather than the positive-heavy resampling used in some prior work. This correction exposes *over-extraction* as the dominant failure mode of zero-shot LLM clause extraction on long contracts, a mode that is suppressed by positive-heavy sampling. The implication is that prior reported precision figures for LLM contract review may overstate enterprise-relevant performance.

The second contribution is empirical. We report what is, to our knowledge, the first cross-family cross-architecture comparison on CUAD that simultaneously varies architecture and frontier-model identity. We evaluate five architectural configurations spanning zero-shot, chain-of-thought, pipeline decomposition, LLM-routed specialists, and a consolidated single-agent ablation, against eleven frontier models from the Anthropic, Google, and OpenAI families, on a fixed sample of approximately 2,999 instances drawn from eighty-one CUAD contracts longer than 100,000 characters. The resulting matrix yields 163,644 sample-level predictions analysed with paired statistical tests.

The third contribution is mechanistic. We identify and name a failure mode specific to pipeline-decomposition multi-agent architectures: the *classification-gate rejection cascade*. When a first-stage classifier overrejects on borderline cases, downstream extractors are never invoked, recall collapses, and the failure is invisible from the perspective of the extractor's own metrics. We document this cascade quantitatively and trace it to a small number of model-family interactions.

The fourth contribution is an architectural critique of the LLM-routed specialist pattern as it appears in current legal multi-agent systems. We observe that on the contract-analysis task, the LLM-router achieves near-perfect routing accuracy (one hundred percent on ten of eleven models in our evaluation), the routed configuration is statistically indistinguishable from a consolidated single-agent prompt on F_2, and the routed configuration produces less clean span outputs. The implication, which we develop in Chapter 6, is that the LLM-routed specialist pattern, in the form most often advocated, can be replaced with a deterministic dictionary lookup without measurable accuracy loss.

The fifth contribution is practitioner-oriented: a six-gap analysis connecting CUAD benchmark performance to enterprise CLM deployment reality. We mention this contribution here without elaboration, as it is developed in Chapter 6.

We emphasise here, to set reader expectations, that the empirical findings of the thesis are not uniformly positive for multi-agent designs. No multi-agent configuration we evaluate consistently outperforms the strongest single-agent baseline on F_2, and the mean F_2 across eleven models for all five configurations falls within a 1.1-point band.

Architectural ranking varies systematically by model family. The thesis frames these results not as an indictment of multi-agent CLM but as a delineation of the conditions under which multi-agent designs help, hurt, or are inert — which is the question Sub-RQ1 explicitly asks.

1.6 Thesis Outline

The remainder of the thesis is organised as follows. Chapter 2 reviews the literature on contract analysis, the evolution from rule-based to neural to LLM-based approaches, multi-agent systems for legal AI, the conditions under which multi-agent designs help or hurt, and evaluation methodology in legal NLP, and positions the present work against this body of evidence. Chapter 3 specifies the methodology, including the five architectural configurations evaluated, the eleven frontier models surveyed, the natural class-distribution sampling design, the metric set, and the statistical testing protocol. Chapter 4 describes the implementation, including the LangGraph-based orchestration [Lan24a], the Langfuse-based observability layer [Lan24b], prompt construction, and reproducibility considerations. Chapter 5 reports the empirical results, the per-hypothesis tests, and the mechanistic diagnosis of the classification-gate rejection cascade. Chapter 6 develops the architectural critique of the LLM-routed specialist pattern, the practitioner-oriented six-gap analysis, and a discussion of threats to validity. Chapter 7 concludes with limitations, implications for enterprise CLM SaaS design, and directions for future work.

2 Literature Review

This chapter situates the thesis within four threads of prior work: the CLM problem space and its enterprise stakes (Section 2.1), the evolution of contract analysis methods from rule-based pipelines to LLMs (Sections 2.2 and 2.3), multi-agent systems and their reception in legal AI (Section 2.4), the conditions under which multi-agent designs help or hurt (Section 2.5), and evaluation methodology in legal NLP (Section 2.6). The chapter closes with a synthesis (Section 2.7) that positions the present work against the gap identified in Chapter 1.

2.1 The CLM Problem Space

Contract Lifecycle Management denotes the end-to-end set of activities through which an organisation authors, negotiates, executes, monitors, renews, and ultimately retires its contractual instruments. Practitioners typically partition the lifecycle into a request phase, an authoring phase, a negotiation and approval phase, an execution phase, and an obligation-management phase, with audit and renewal looping back into authoring. From an information-systems perspective, CLM is therefore a long-running, multi-stakeholder workflow that sits at the intersection of legal, procurement, finance, and operational functions.

The economic stakes are large. World Commerce & Contracting estimates that organisations forfeit value equivalent to roughly nine percent of annual revenue through poor contracting practices on average, with the figure rising substantially in complex industries [Wor25]. The same source reports that contract-related data is dispersed across an average of twenty-four systems within a single organisation, and that the majority of executives surveyed regard their contracts as too rigid to accommodate change [Wor25]. Goldman Sachs analysts estimate that forty-four percent of legal-task time in the United States is technically exposed to automation by generative AI, second only to administrative occupations [Hat+23].

The bottleneck within this lifecycle is typically the review stage, in which contracts must be parsed against playbooks to identify present clauses, missing clauses, deviations from preferred wording, and risk-bearing provisions. Three sub-tasks dominate. *Clause extraction* locates and bounds the textual span that instantiates a given clause category — for example, an indemnification clause or a change-of-control provision. *Risk*

identification classifies extracted clauses against playbook standards, flagging deviations that warrant escalation. *Obligation tracking* converts executed clauses into a structured calendar of duties, milestones, and contingent triggers. Each sub-task amplifies the cost of error: a missed indemnification clause produces audit-time financial exposure, a misclassified risk produces negotiation-time leverage loss, and an untracked obligation produces post-execution non-compliance.

These three sub-tasks share a structural property that motivates the rest of this chapter. They are all, at their core, span-extraction and span-classification problems over long, heterogeneous, domain-specific documents — exactly the regime in which information-extraction methodology has evolved over the past two decades.

2.2 From Rule-Based to Neural Approaches

Pre-LLM contract analysis was dominated by symbolic and statistical methods. Rule-based extractors, often built around regular expressions, gazetteers, and hand-engineered grammars, achieved high precision on narrow clause types but degraded sharply on contracts whose drafting style departed from the templates encoded in the rules. The transition to statistical sequence labelling, exemplified by Conditional Random Fields [LMP01], provided a principled probabilistic treatment of span boundaries and substantially improved cross-document robustness. The CRF formulation was widely adopted for legal information extraction in the 2000s and remained competitive through the deep-learning transition.

The deep-learning era introduced neural sequence labellers that combined bidirectional recurrent encoders with CRF output layers. Lample et al. demonstrated that bidirectional LSTM encoders coupled with a CRF decoding layer outperformed prior named-entity recognition systems without hand-engineered features, establishing the BiLSTM-CRF architecture as a workhorse for span extraction in legal and biomedical domains [Lam+16]. The introduction of the Transformer encoder and BERT [Dev+19] initiated a further shift, replacing recurrent encoders with self-attention and enabling effective transfer from large general-domain corpora to small legal corpora through fine-tuning.

Domain-specific pre-training emerged in response to the observation that legal text exhibits lexical, syntactic, and discourse properties that differ systematically from general-purpose web text. LEGAL-BERT, introduced by Chalkidis et al., investigated three adaptation strategies: out-of-the-box use, additional pre-training on legal corpora, and pre-training from scratch, and reported that domain pre-training generally improved downstream legal NLP performance, with pre-training from scratch giving the largest gains in the most domain-shifted tasks [Cha+20]. LEGAL-BERT and its

descendants became standard baselines for legal classification and extraction tasks throughout the late 2010s and early 2020s.

Three limitations of the rule-based and BERT-era stack motivated the transition to LLMs. First, span-extraction supervision is expensive: legal annotation requires lawyer-time, and the expert-annotated CUAD dataset, with its five hundred contracts and forty-one categories [Hen+21], was a large undertaking by the standards of legal NLP. Second, BERT-class encoders are limited by their context windows, typically 512 tokens, which forces sliding-window inference over long contracts and complicates document-level reasoning. Third, BERT-class models do not generalise zero-shot to new clause categories: each new category requires labelled examples, which is incompatible with the long tail of clause types that arises in production CLM. These limitations created the opening that LLM-based contract analysis has, in the past four years, attempted to fill.

2.3 LLMs in Contract Analysis

The CUAD dataset of Hendrycks et al. is the foundational benchmark for clause-level contract analysis. It comprises 510 commercial contracts sourced from EDGAR filings, with 13,000 expert annotations across forty-one clause categories [Hen+21]. The original baselines, fine-tuned RoBERTa- and DeBERTa-class encoders, achieved best results in the regime of forty-four percent precision at eighty percent recall (AUPR-based reporting), which the authors interpreted as evidence of a substantial capability gap in 2021. CUAD remains the most widely cited public benchmark for contract clause extraction and underpins the evaluation in this thesis.

Two complementary datasets have shaped subsequent work. ContractNLI, introduced by Koreeda and Manning, recasts a subset of contract review as document-level natural language inference: given a non-disclosure agreement and a hypothesis drawn from a fixed set of seventeen, the system must label entailment, contradiction, or neutrality, while additionally identifying evidence spans [KM21]. MAUD, introduced by Wang et al., focuses on merger agreements and provides reading-comprehension supervision derived from the American Bar Association's Public Target Deal Points Study, with 150 agreements and roughly 39,000 examples [Wan+23a]. LEDGAR, introduced by Tuggener et al. [Tug+20], scales further along the multi-label classification axis with approximately 100,000 provisions extracted from 60,000 contracts, at the cost of weaker supervision quality. These datasets together provide the empirical surface against which LLM-based contract analysis is currently evaluated.

The arrival of GPT-4-class models prompted a wave of human-comparison studies. The most-cited example is Martin et al.'s "Better Call GPT" study, which compared

frontier LLMs against junior lawyers and legal-process outsourcers on issue identification and issue location in commercial contracts [Mar+24]. The study reports that GPT-4-1106 attained an issue-determination F-score of 0.87, tied with legal-process outsourcers and within one point of junior lawyers, while completing review in under five minutes per document against the human comparators' forty-three to fifty-six minutes. The study additionally reported a 99.97% cost reduction. Although the study is vendor-funded and its quantitative claims should therefore be received critically, the directional finding — that frontier LLMs are competitive with junior human reviewers on standardised contract-review tasks — is consistent with subsequent independent evidence.

The most recent and methodologically careful evaluation of LLMs on CUAD is ContractEval, by Liu et al. [Liu+25]. ContractEval evaluates nineteen LLMs (four proprietary and fifteen open-source) on clause-level legal-risk identification across forty-one categories, sampling 4,128 data points from 102 CUAD contracts. The paper reports four findings of direct relevance to the present thesis. First, proprietary models outperform open-source models on both correctness and effectiveness metrics. Second, scaling open-source models yields diminishing returns. Third, "thinking" or reasoning-mode prompting improves output effectiveness — measured by Jaccard similarity between predicted and reference spans — but degrades correctness, suggesting that extended reasoning over-complicates simple extraction decisions. Fourth, open-source models exhibit a *laziness* failure mode in which they default to "no related clause" responses even when the relevant text is present in the contract; ContractEval quantifies this with a laziness rate metric. The paper additionally argues for F_2 alongside F_1, on the grounds that legal-risk asymmetries make missed clauses costlier than spurious clauses.

ContractEval is the closest predecessor to the present thesis in scope and method, and we adopt three of its design choices: F_1 and F_2 as primary correctness metrics, Jaccard similarity as an effectiveness metric, and explicit accounting of the laziness or wrong-rejection rate. We depart from ContractEval in two respects. First, we evaluate *architectures* rather than *models alone*, fixing the family of LLMs and varying the orchestration around them. Second, we sample at the natural class distribution of CUAD (approximately thirty percent positive) rather than at the more positive-heavy ratio that some evaluations use, which we argue exposes over-extraction more honestly.

Three persistent limitations of LLM-based contract analysis remain salient regardless of evaluation choice. The first is hallucination: LLMs occasionally produce extracted spans that are not present in the source contract, a behaviour that is unacceptable in legal review and has motivated retrieval-grounded designs [Cui+24; Rap+25]. The second is prompt sensitivity: minor, intent-preserving changes in the prompt can produce substantial changes in output, as quantified by Chatterjee et al.'s POSIX

prompt-sensitivity index [Cha+24]. The third is the over-extraction / under-extraction tradeoff: zero-shot models tuned to maximise recall on positive-heavy samples can over-extract on natural distributions, while models tuned to avoid spurious extractions can under-extract on rare categories. These three limitations frame the multi-agent intervention.

2.4 Multi-Agent Systems for Legal AI

Multi-agent LLM systems decompose a task across multiple LLM invocations that play differentiated roles and communicate through structured messages. The motivation, common to general-purpose multi-agent frameworks such as AutoGen [Wu+23], MetaGPT [Hon+24], CAMEL [Li+23], and Mixture-of-Agents [Wan+24], rests on three claims: that role specialisation reduces the per-step prompt complexity each agent must handle, that decomposition exposes intermediate reasoning to inspection and intervention, and that ensembling across agents recovers errors that any single agent would commit. Reflexion [Shi+23] and Tree-of-Thoughts [Yao+23a] add iterative-refinement and search-style structure on top of single agents, while ReAct [Yao+23b] interleaves reasoning with tool calls, and Self-Consistency [Wan+23b] aggregates multiple stochastic completions. Generative Agents demonstrate that long-running social simulations can be sustained by LLM agents with memory and reflection [Par+23]. Wei et al.'s Chain-of-Thought prompting [Wei+22] sits beneath all of these as the within-agent reasoning primitive.

In the legal domain, three multi-agent systems are most directly relevant to this thesis. ChatLaw, in its 2024 revision by Cui et al., reformulates the original retrieval-augmented Chinese legal LLM [Cui+24] as a multi-agent collaborative legal assistant with knowledge-graph-enhanced mixture-of-experts and standardised operating procedures modelled on law-firm workflows; the paper reports a 7.73-point improvement over GPT-4 on Lawbench. PAKTON, by Raptopoulos et al., is a fully open-source multi-agent framework for question answering in long legal agreements, structured around three specialised agents: an *Archivist* that handles parsing and embedding with hierarchical document trees, a *Researcher* that performs hybrid graph-aware retrieval, and an *Interrogator* that performs multi-step iterative reasoning in a ReAct style [Rap+25]. PAKTON was accepted as an oral presentation at EMNLP 2025 and is benchmarked on ContractNLI and LegalBench-RAG. L-MARS, by Wang and Yuan, organises legal question answering as a four-agent workflow comprising query decomposition, agentic search across BM25 and case-law APIs, judge-style verification of jurisdictional and temporal validity, and final summarisation [WY25]; the system reports a 38-point improvement over zero-shot single-agent on its LegalSearchQA benchmark.

System	Domain	Topology	Routing	Verification
ChatLaw v2 [Cui+24]	Legal QA (Chinese)	Hierarchical	SOP-based	MoE consensus
PAKTON [Rap+25]	Legal QA/contracts	Pipeline	Static	ReAct iterative
L-MARS [WY25]	Legal QA/search	Pipeline	Static	Judge agent
AutoGen [Wu+23]	General	Peer	Conversational	None (default)
MetaGPT [Hon+24]	Software dev.	Hierarchical	SOP-based	Review agent
MoA [Wan+24]	General	Layered ensemble	Broadcast	Aggregator

Table 2.1: Representative multi-agent systems by topology, routing, and verification structure.

These three systems are valuable demonstrations that multi-agent legal architectures can be assembled, but each evaluates a single architectural choice on a single benchmark and does not ablate the multi-agent design itself. The empirical question of which multi-agent pattern is preferable for clause-level contract analysis under a fixed model and task remains underdetermined by this body of work.

The taxonomic axes along which these systems differ are nevertheless relevant for the present thesis. Architectures vary in *topology* (hierarchical with an orchestrator versus peer with mutual messaging), in *routing* (static pipelines versus dynamic LLM-mediated routing versus dictionary-based routing), in *specialisation granularity* (one specialist per task type versus one specialist per clause category versus one specialist per reasoning step), and in *verification structure* (single pass versus reflective revision versus multi-agent debate). Table 2.1 summarises a representative selection of multi-agent legal and general systems along these axes.

The reported gains in this body of work, while real, must be interpreted carefully. ChatLaw v2's 7.73-point improvement is reported relative to GPT-4 on Lawbench, not against a comparable multi-agent ablation; the 38-point improvement reported by L-MARS is computed against zero-shot single-agent on a benchmark whose construction is itself part of the contribution. Without architectural ablations, these gains conflate the multi-agent design choice with retrieval, prompting, and benchmark-construction choices made in the same systems.

2.5 When Multi-Agent Helps and When It Does Not

A small but rapidly growing body of recent work has begun to interrogate the conditions under which multi-agent designs improve on single-agent designs. Three contributions are central to the framing of the present thesis.

Gao et al. ask, in a paper titled "Single-agent or Multi-agent Systems? Why Not

Both?", whether the empirical advantage of multi-agent systems persists as base-model capability improves [Gao+25]. Their answer, supported by experiments on BigCodeBench, SWE-Bench, and the MAST trace set, is that the advantage diminishes for frontier models such as o3 and Gemini 2.5 Pro, whose improved long-context reasoning, persistent memory, and tool-use absorb the workload that previously required decomposition. The authors propose hybrid architectures that localise the error-prone agent inside an otherwise single-agent design. The implication for the present thesis is that whether multi-agent helps in CLM may depend on the strength of the underlying model.

Cemri et al., in "Why Do Multi-Agent LLM Systems Fail?", construct the Multi-Agent System Failure Taxonomy (MAST) by analysing more than 200 tasks and 1,600 traces across seven multi-agent frameworks including AutoGen, MetaGPT, ChatDev, and CAMEL [Cem+25]. They identify fourteen failure modes grouped into three categories: specification and system-design failures (such as disobedience to role specification and rigid step repetition), inter-agent misalignment failures (such as conversation-history loss and goal drift), and verification-and-termination failures. The authors report that simple interventions — clearer role specification, tighter orchestration — yield only modest gains, of the order of fourteen to fifteen percent, suggesting that multi-agent failures arise from fundamental design rather than from prompt-level tuning.

Bogavelli et al. introduce AgentArch, a benchmark that evaluates eighteen agent configurations across six frontier LLMs on enterprise workflow tasks [Bog+25]. They report two findings of direct relevance. First, no single agent architecture is universally optimal across model families: smaller models can outperform larger models when matched to an appropriate architecture, and the architecture that works best for one family does not necessarily work best for another. Second, function-calling prompting outperforms ReAct-style prompting in multi-agent settings, where ReAct produces hallucinated tool calls. Absolute performance is also reported as low: best-of-class models reach only 35.3% on the harder enterprise task and 70.8% on the easier one.

Reading these three contributions together with Sanwal's Layered Chain-of-Thought paper [San25], which proposes external verification at each reasoning layer to address the unverified-intermediate-step problem of vanilla CoT, and with the failure-mode evidence from ContractEval [Liu+25], a coherent picture emerges. Multi-agent designs are not a free lunch. They help when the task is decomposable, when individual specialists can be made meaningfully better than a generalist, when the orchestration overhead does not introduce its own failure modes, and when the underlying model is weak enough that decomposition still offers a usable lever. They hurt when these conditions do not hold. The empirical task of the present thesis is to determine which of these conditions hold for clause-level contract analysis on CUAD.

2.6 Evaluation Methodology in Legal NLP

Evaluation methodology in legal NLP, particularly for clause-level extraction, has not converged on a single protocol, and the choices made in published evaluations materially affect the rankings they produce. We treat four methodological axes here: metric selection, statistical testing, prompt-sensitivity treatment, and sampling design.

Metric selection. Token-level F_1 remains the default for span extraction, but the legal-risk asymmetry between false negatives (missed clauses, with downstream financial and compliance exposure) and false positives (spurious clauses, recoverable at human review) has motivated the use of F_β with $\beta > 1$. ContractEval explicitly adopts F_2 alongside F_1 on this rationale and additionally reports Jaccard similarity as a measure of span effectiveness [Liu+25]. We follow this convention. The choice is not innocuous: F_2 favours architectures that retrieve aggressively, and an architecture that ranks first on F_2 may rank last on precision.

Statistical testing. Dror et al.'s "Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing" [Dro+18] remains the canonical reference for significance testing in NLP. The guide argues that paired tests should be used when systems are evaluated on the same instances, that test selection should reflect distributional assumptions, and that multiple-comparison correction should be applied when many systems are compared. For paired binary outcomes — system A correct, system B incorrect, and the reverse — McNemar's test [McN47] is the standard. For continuous metrics on small samples, paired bootstrap is preferred. The present thesis applies McNemar for binary correctness and paired bootstrap for F_2 differences, with Holm-style correction for the family of comparisons under each hypothesis.

Prompt sensitivity. Chatterjee et al.'s POSIX index [Cha+24] formalises the observation that LLM outputs change under intent-preserving prompt perturbations such as paraphrasing, formatting changes, and template substitutions. They report that increased parameter count and instruction tuning do not necessarily reduce sensitivity, and that a single few-shot exemplar typically reduces sensitivity substantially. The implication for legal evaluation is that single-prompt comparisons may be unstable, and the architectural ranking between systems can swap under prompt perturbation. We treat prompt sensitivity as a methodological constraint: we fix prompt templates per architecture across models, document the prompts in full, and discuss prompt sensitivity as a threat to validity in Chapter 6.

Sampling design. The most consequential methodological choice in CUAD-based evaluation is the sampling ratio between clause-present and clause-absent instances. CUAD's natural distribution at the clause-category level is heavily imbalanced toward absences. Evaluations that resample to a more positive-heavy ratio — for example, two-thirds positive — produce optimistic precision estimates because the prior probability of a clause being present is artificially elevated. Evaluations that hold the ratio at the natural level expose over-extraction more honestly but risk small-sample variance on rare categories. We adopt the natural-distribution sampling design at approximately thirty percent positive, holding the same sample across architectures and models so that comparisons are paired at the instance level.

The combination of metric, statistical, sensitivity, and sampling choices defines a protocol space, and protocols differ substantially across published evaluations. The present thesis fixes a single protocol and applies it uniformly across all eleven models and five architectures, and reports the protocol explicitly so that other workers can replicate or contest it.

2.7 Synthesis and Research Positioning

The literature surveyed in this chapter establishes four facts that motivate the specific design of the present thesis. First, contract analysis is an enterprise-critical task with a substantial economic surface, an established benchmark in CUAD [Hen+21], and a recent baseline-establishing evaluation in ContractEval [Liu+25]. Second, multi-agent systems for legal AI exist and are advocated, with notable systems such as ChatLaw [Cui+24], PAKTON [Rap+25], and L-MARS [WY25], but each evaluates a single architecture and does not ablate the multi-agent design choice. Third, recent cross-domain evidence — Gao et al. [Gao+25], Cemri et al. [Cem+25], and Bogavelli et al. [Bog+25] — challenges the assumption that multi-agent designs are universally beneficial, and identifies model-architecture interaction, fourteen named failure modes, and a diminishing-returns regime as serious caveats. Fourth, evaluation methodology in legal NLP is heterogeneous, and the choice of metric, statistical test, and sampling ratio materially shapes the conclusions one can draw.

The gap that this thesis addresses is the absence of a rigorous, paired, statistically tested comparison of multi-agent architectural patterns for clause-level contract analysis on CUAD, conducted across multiple frontier-model families, under a natural class distribution, and with explicit mechanistic diagnosis of failure modes. Sub-RQ1 asks which architectural pattern is most effective; Sub-RQ2 asks how multi-agent compares to single-agent on accuracy and cost; Sub-RQ3 asks which architectural decisions enable auditable reasoning traces under the EU AI Act [EC24] and GDPR [EC16] regimes. The

hypotheses H1, H2, and H3, framed in Chapter 1, operationalise the comparison.

The methodology developed in Chapter 3 instantiates this protocol: five architectural configurations spanning zero-shot, chain-of-thought, pipeline decomposition, LLM-routed specialists, and consolidated specialist prompts; eleven frontier models from three families; the CUAD benchmark restricted to long contracts; natural class-distribution sampling at approximately thirty percent positive; F_1, F_2, and Jaccard correctness and effectiveness metrics; and McNemar and paired-bootstrap statistical tests with multiple-comparison correction. Chapter 4 specifies the implementation, including the LangGraph-based orchestration [Lan24a] and Langfuse-based observability layer [Lan24b] that secure the auditable-trace property addressed by Sub-RQ3. Chapter 5 reports the results, which, to anticipate, do not uniformly favour multi-agent designs and instead reveal a model-family-dependent ranking, a classification-gate rejection cascade specific to pipeline decomposition, and a near-complete equivalence between LLM-routed specialists and consolidated specialist prompts. The interpretation of these findings, and their implications for enterprise CLM SaaS design, is taken up in Chapters 6 and 7.

3 Methodology

This chapter specifies the experimental design used to evaluate multi-agent architectures for clause extraction in Contract Lifecycle Management (CLM). The chapter is organised around the research questions introduced in Chapter 1 and the conceptual framing developed in Chapter 2: we contrast a small set of architectural configurations on a fixed legal-NLP benchmark, holding model identity and prompt content constant within each comparison, and use paired statistical procedures to decide between competing hypotheses about architectural value. Where Chapter 2 surveyed the design space of agentic systems and the empirical record of LLMs on contract review, this chapter operationalises that space into testable configurations and a falsification protocol.

3.1 Research Design

The study follows a controlled comparative design in the spirit of [Dro+18] and [Car+20]: a fixed evaluation set, a fixed metric family, and a small set of clearly-distinguished system configurations whose differences are intentionally minimised to a single architectural axis at a time. Each configuration is run, end to end, against the same sample of contracts using the same eleven LLMs, so that any observed performance difference can be attributed either to architecture, to the model, or to their interaction, but not to dataset drift or prompt contamination.

We evaluate five configurations. Two are single-agent baselines: **B1**, a zero-shot replication of the ContractEval prompt of [Liu+25] delivered as a single user message; and **B4**, a chain-of-thought (CoT) variant that elicits step-by-step reasoning and a structured final answer. Three are multi-agent configurations: **M0**, a classify-then-extract pipeline in which a binary classifier first decides whether the target category is present and an extractor is invoked only conditionally; **M1**, an orchestrator with three domain specialists (Risk/Liability, Temporal/Renewal, IP/Commercial) and an LLM router; and **M6**, a single-agent ablation that consolidates the union of all specialist instructions into one prompt without routing or pipelining. M6 is the architectural control that isolates the contribution of *routing and decomposition* from the contribution of *specialist domain knowledge expressed in prompts*.

Three retrieval and verification configurations originally proposed in our pre-registration (BM25 baseline B5, BM25+reranker B6, extract-then-verify M7, and an ensemble M8)

were cut from the main study to keep the comparison space tractable; we discuss them as future work in Chapter 7. Holding the design to the five configurations above preserves the single-variable ablation principle: M0 vs. B1 isolates the value of a classification gate; M1 vs. M6 isolates the value of routing and decomposition given equivalent prompt content; B4 vs. B1 isolates the value of explicit reasoning under a single agent.

Each (configuration, model) cell yields one official run, for a total of $5 \times 11 = 55$ official runs. Within a run the model evaluates one sample at a time across all 41 CUAD categories. We log structured traces (Section 4.5) sufficient to reconstruct every per-call decision *post hoc*.

3.2 Dataset Selection and Justification

We use the Contract Understanding Atticus Dataset [Hen+21] as the evaluation corpus. CUAD is the largest publicly available expert-annotated benchmark for commercial-contract clause extraction; it contains 510 contracts with 41 categories of legally salient provisions and is the benchmark used by ContractEval [Liu+25] and the majority of recent LLM-based contract review work surveyed in Chapter 2. Its annotation is performed by attorneys, its license permits research evaluation, and its category taxonomy aligns with the categories that legal-tech CLM platforms surface in production review workflows.

For this thesis we restrict evaluation to the 81 CUAD contracts longer than 100,000 characters. Two reasons motivate this filter. First, long-context performance is the regime in which agentic decomposition is hypothesised to add value (Chapter 2): short contracts can be ingested in a single prompt by every model in our study, so any architectural differentiation should be most visible on long ones. Second, long contracts approximate the operational distribution of CLM platforms more closely than short addenda or one-page NDAs that dominate the lower tail of CUAD. The 81-contract subset yields a raw enumeration of $81 \times 41 = 3{,}321$ (contract, category) pairs per evaluation pass. After per-run downsampling (Section 3.3) each run produces approximately 2,999 predictions; aggregated across the 55 official runs we analyse 163,644 sample-level predictions after dropping 1,297 samples ($\approx 0.79\%$) for which the model returned a malformed classification token (Section 3.6).

Following CUAD's evaluation convention, ground truth is the union of attorney-annotated answer spans for each (contract, category) pair. A category for which the annotators marked no span is a true negative (the clause is genuinely absent). This convention preserves the natural imbalance of the corpus, which we discuss next.

3.3 Sampling Strategy: Natural Class Distribution

CUAD is naturally imbalanced toward absence: most categories are absent from most contracts. Pilot work for this thesis initially used an `include_negative=True` configuration that produced an approximately 67% positive / 33% negative split per run. We replaced this with a `neg_ratio=0.7` sampler that produces a 30% positive / 70% negative split, mirroring the natural class prior in the long-contract subset. Switching to the natural prior makes one failure mode visible that the inflated-positive sampler had masked: under a positive-heavy sample, over-extraction (a model emitting text for a category that is in fact absent) is rarely penalised because there are few absent samples; under the natural prior, over-extraction directly depresses precision and is reflected in F-measures. We retain the natural-prior sampler for all reported results.

Each run draws samples with a fixed pseudo-random seed (`seed=42`) so that the same (contract, category) pairs are evaluated across configurations within a model. This is the basis for paired comparison: every statistical test is computed on the intersection of sample IDs across the two configurations being compared, never on independent draws. Where partial runs differ in coverage (rare), the intersection is taken explicitly and reported.

To assess whether architectures behave differently on rare versus common provisions, we partition the 41 CUAD categories into three prevalence *tiers* computed from the natural positive rate of each category in the long-contract subset: *common*, *moderate*, and *rare*. The tier cutpoints are fixed before any model is run and are the same across all configurations and models. Per-tier metrics (Section 3.4) are then reported alongside the pooled metric.

3.4 Metric Selection

Clause extraction has two natural granularities. At sample level, it is a binary classification problem (*is the clause present in this contract?*). At span level, it is an extraction problem (*which substring expresses the clause?*). We report metrics at both granularities.

The primary metric is the F_β score with $\beta = 2$, which weights recall four times as heavily as precision. The choice reflects the asymmetry of legal review: a missed risky clause is a substantive harm to the downstream user (an attorney or contracting party), whereas a falsely flagged clause is corrected cheaply by the reviewer in the next stage of the CLM workflow. This recall-favouring asymmetry is consistent with how earlier CUAD work has framed evaluation [Hen+21; Liu+25], and we treat F_2 as the default for hypothesis testing.

We also report standard F_1 as a comparison-friendly secondary metric, precision

and recall separately to expose the failure mode behind any F_β change, and span-level Jaccard overlap to capture the quality of extracted spans conditional on extraction. Two complementary diagnostic rates are tracked: the *laziness rate*, defined as the fraction of clause-present samples for which the model output is "no related clause" (i.e. silent omission), and its dual, the *over-extraction rate*, defined as the fraction of clause-absent samples for which the model returned a non-empty span (i.e. hallucinated or over-eager flagging). All metrics are reported pooled across the 41 CUAD categories and broken down per tier (common, moderate, rare).

3.5 Hypotheses and Success Criteria

We pre-register three hypotheses corresponding to the three sub-research-questions of Chapter 1. Each is stated neutrally as a falsifiable proposition; the chapter does not pre-commit to a result.

H1 (Architectural value). *There exists at least one multi-agent configuration $C \in \{M0, M1, M6\}$ that outperforms the strongest single-agent baseline $B^\star \in \{B1, B4\}$ on F_2.* The null is that no such C exists. The test statistic is McNemar's discordant-pair count between $B^\star$ and C, computed on per-sample classification correctness (TP $\cup$ TN treated as "correct"); we use the exact binomial form via `scipy.stats.binomtest` [McN47; Vir+20]. The decision rule is rejection of the null if the BH-corrected two-sided p-value falls below $\alpha = 0.05$ on at least one such pair, with the F_2 direction favouring C. The effect size reported alongside is the per-model paired bootstrap mean ΔF_2 with 300 resamples and a 95% percentile confidence interval [Efr79].

H2 (Tier-equity). *Multi-agent architectures reduce the rare–common F_2 gap relative to single-agent baselines.* Define $\mathrm{Gap}(C) = F_2^{\mathrm{common}}(C) - F_2^{\mathrm{rare}}(C)$ for configuration C. The null is $\mathrm{Gap}(C_{\mathrm{multi}}) \geq \mathrm{Gap}(B^\star)$. Inference uses a per-tier paired bootstrap on the gap difference $\Delta\mathrm{Gap} = \mathrm{Gap}(C_{\mathrm{multi}}) - \mathrm{Gap}(B^\star)$ with 300 resamples and 95% percentile CI; the null is rejected if the upper end of the CI lies below zero.

H3 (Architecture vs. prompts). *The benefit (if any) of M1 over a single-agent baseline is attributable to the architectural decomposition itself, beyond what is achievable by consolidating specialist instructions into a single prompt (M6).* The null is that M1 and M6 are statistically indistinguishable. The primary test is two-sided McNemar's exact on M1 vs. M6 per-sample correctness; a paired-bootstrap ΔF_2 confidence interval is reported as effect size. As a secondary test we compare M0 vs. M6 to separate the contribution of the

classification gate from that of decomposition. Span-level Jaccard is reported alongside as a quality-of-output check.

For all three hypotheses we report pooled and per-tier statistics; H1 and H3 use BH-corrected p-values within their family of pairwise comparisons (Section 3.6). The pre-registered family for H1 is the ten pooled pairwise comparisons induced by the five configurations (i.e. $\binom{5}{2} = 10$). H2 is reported as effect-size CIs without an additional FDR correction because it is a single directional contrast per (configuration, baseline) pair on a different statistic than H1.

3.6 Statistical Methodology

Per-sample paired tests. For each pair of configurations being compared, we compute per-sample correctness as the indicator that the model's binary classification matches the ground truth (TP or TN). We then form the McNemar discordant pair counts b (correct under A, incorrect under B) and c (incorrect under A, correct under B) and obtain the two-sided exact p-value as the binomial tail probability under $\text{Binomial}(b + c, 0.5)$ [McN47], implemented via `scipy.stats.binomtest` [Vir+20]. The exact binomial form is preferred over the chi-squared approximation because some discordant counts on rare-category subsets are small.

Paired bootstrap. For continuous metrics (F_2, F_1, Jaccard, gap differences) we use the paired non-parametric bootstrap [Efr79; RM05]. Sample IDs (not categories) are resampled with replacement; both configurations are evaluated on the resampled IDs to preserve pairing. We use $B = 300$ resamples and report the 95% percentile confidence interval. Per-tier bootstraps for H2 resample within tier strata. The number of resamples (300) was chosen to balance Monte Carlo error against compute cost across 55 runs; we report bootstrap stability as a robustness check in Chapter 5.

Multiple-comparison correction. The H1 family is the ten pooled pairwise configuration comparisons, evaluated within each model. We apply the Benjamini–Hochberg procedure [BH95] at $\alpha = 0.05$ to control the false discovery rate within this family. We report both raw and BH-corrected p-values to allow the reader to apply alternative thresholds.

Paired comparison protocol. Every test is computed on the intersection of sample IDs between the two configurations, never on independent samples. When a model's run is partially missing for some sample IDs (e.g. because of provider rate limits), the

missing samples are dropped *symmetrically* from both configurations being compared. The intersection size is reported with each test.

Missing-data handling. Some model outputs return a malformed classification token (neither a clean "yes/no" nor a parseable span). Such samples cannot be assigned a TP/FP/TN/FN label and are excluded from per-sample tests. Across the entire study this affects 1,297 of 164,941 predictions, or 0.79%. The exclusion is applied symmetrically per pairwise comparison: if a sample is null in either of the two configurations being compared, it is dropped from that comparison only. This avoids penalising the configuration that happened to produce a parseable answer.

Statistical power. [Car+20] caution that NLP comparisons are frequently underpowered. The pairwise sample size after intersection is on the order of 2,900–3,000 per (configuration, model) pair, which is comparable to or larger than the GLUE test sets analysed in their meta-study and is sufficient to detect small-to-moderate effects in F_2 with the bootstrap procedure used here. We do not, however, claim power against arbitrarily small effects; the smallest effect size detectable in our design is reported alongside each comparison.

3.7 Model Selection

We evaluate eleven frontier proprietary models drawn from three families: Anthropic (Claude Haiku 4.5, Claude Sonnet 4, Claude Sonnet 4.6, Claude Opus 4.6); Google (Gemini 2.5 Flash, Gemini 2.5 Pro, Gemini 3 Flash, Gemini 3.1 Pro); and OpenAI (GPT-4.1, GPT-4.1 Mini, GPT-4.1 Nano). The selection covers three deployed model families and, within each family, spans a deliberate compute-tier range from light (Haiku, Flash, Nano) to heavy (Opus, Pro). The breadth lets us separate effects that are model-family-specific (e.g. a particular family responding well to chain-of-thought) from effects that are architectural in origin. We restrict the main analysis to proprietary models because they represent the deployment substrate of current commercial CLM platforms, which is the population to which the thesis intends to generalise; an open-weights extension is discussed in Chapter 7.

3.8 Threats to Validity

Internal validity. Configuration parity is the central internal-validity concern. We mitigate it by holding the underlying CUAD loader, sampling seed, prompt registry, and trace schema constant across configurations, and by isolating prompts: the same

specialist instructions appear in M1 (split across three agents) and M6 (concatenated in one agent), so the textual prompt content is the same union in both cases. Temperature and decoding parameters are fixed per provider. The chain-of-thought parsing logic for B4 went through one revision (an early delimiter-based parser was replaced by structured-output enforcement; see Section 4.2), and only the post-fix version is included in the official runs.

External validity. The evaluation is single-language (English), single-jurisdiction (predominantly U.S. commercial contracts), and single-corpus (CUAD). Generalisation to other legal traditions, languages, or contract types is not guaranteed by these results and is discussed explicitly in Chapter 7. We also use a fixed snapshot of each model; provider model updates can shift behaviour, and we record exact model identifiers in the run metadata.

Construct validity. F_2 is a proxy for legal review utility, not a measurement of it. The legal-risk asymmetry that motivates $\beta = 2$ is plausible but not measured against actual reviewer behaviour, and reviewer cost of inspecting an over-extracted span is workflow-dependent. We mitigate this by reporting precision, recall, Jaccard, laziness, and over-extraction independently so a reader with a different utility model can re-weight.

Statistical validity. Three concerns deserve flagging. First, multiple comparisons inflate false-positive risk; we control the family-wise FDR via BH [BH95] but not all secondary contrasts are corrected, and we report this transparently. Second, model identity is treated as a fixed factor rather than a random effect, which limits inference to the eleven models studied. Third, bootstrap percentile CIs are known to under-cover for small samples and skewed statistics [Efr79]; we report bootstrap diagnostics in Chapter 5 and treat tail-area p-values as decisive only when corroborated by the McNemar exact test.

4 System Design and Implementation

This chapter documents the system that operationalises the configurations of Chapter 3. The objective is reproducibility: the architectures are described at a level of detail sufficient for an independent team to re-implement them, and the supporting infrastructure (prompt registry, model client, observability layer, dashboard, and test suite) is described at the same level. Code references in this chapter point to the thesis artifact released alongside the document.

4.1 Architectural Patterns Evaluated

The five configurations populate three orthogonal axes of the agentic design space: *number of stages* (single vs. multi-stage), *routing* (none, static, or LLM-driven), and *specialisation* (a single generalist agent vs. several domain specialists vs. a consolidated specialist prompt). Table 4.1 summarises the five configurations and their position in this space.

ID	Name	Stages	Routing	Specialisation
B1	Zero-shot single agent	1	none	generalist
B4	Chain-of-thought single agent	1	none	generalist + reasoning
M0	Classify-then-extract pipeline	2	static (gate)	generalist
M1	Orchestrator + 3 specialists	2	LLM router	three domain specialists
M6	Consolidated specialist (ablation)	1	none	one agent, union of specialist prompts

Table 4.1: The five evaluated configurations across the three design axes.

4.2 Baseline Architectures

B1 — ContractEval replication. B1 is a faithful replication of the zero-shot prompt protocol of [Liu+25]. The model receives a single user message containing the contract text, the target category name, the category indicator (a short attorney-style description of what the category covers, taken from CUAD's annotation guidelines), and an explicit instruction to either return the verbatim span(s) of the clause or the literal string "no

related clause". The system message is identical across categories. No examples, no chain-of-thought instructions, and no auxiliary structure are added. B1 establishes the floor for a competent single-agent reading.

B4 — Chain-of-thought single agent. B4 retains B1's task framing but explicitly elicits step-by-step reasoning followed by a structured final answer. The user prompt instructs the model to reason about the category definition, scan the contract for candidate spans, weigh ambiguous matches, and only then emit a final `<answer>...</answer>` block containing either verbatim span(s) or "no related clause".

The B4 prompt went through one substantive revision during pilot work. The original delimiter-based parser extracted any text occurring after a sentinel string, which under certain models produced spurious matches: when the chain-of-thought monologue itself quoted contract passages, the parser harvested them as if they were the final answer, inflating the over-extraction rate. The fix was to enforce a structured-output contract: the parser now requires the final block to be enclosed in an explicit answer tag and rejects the trace otherwise, with a small repair step that re-prompts the model for a structured emission if the tag is missing. Only the post-fix version is in the official runs; the parsing-bug runs are retained for reference but excluded from all reported statistics.

4.3 Multi-Agent Architectures

M0 — Classify-then-extract pipeline. M0 decomposes the per-category task into two LLM calls. The first call is a binary *classifier* agent that receives the contract and the category and returns a single yes/no token indicating whether any span of the category is present. The second call is an *extractor* agent that is invoked only when the classifier returns yes; it receives the same inputs and produces the verbatim span(s). When the classifier returns no, the extractor is bypassed and the system records "no related clause" as the final answer. The two agents share a category indicator but have disjoint prompts: the classifier's prompt explicitly forbids span emission, and the extractor's prompt assumes presence. M0 implements the simplest form of conditional computation in the design space and is the architectural reference for the value of a classification gate.

M1 — Orchestrator with three specialists. M1 consists of an *orchestrator* that receives the contract and the category and routes the request, via an LLM router prompt, to one of three domain specialists. The router prompt enumerates the three specialists, gives a

one-line description of each, and asks for a single category-name decision. The three specialists partition the 41 CUAD categories as follows:

- **Risk/Liability specialist** (13 categories): liability caps, indemnification, insurance, warranty, governing law, and similar risk-allocation provisions.
- **Temporal/Renewal specialist** (11 categories): term, renewal, termination, notice periods, post-termination obligations, and other time-bounded provisions.
- **IP/Commercial specialist** (17 categories): IP ownership and licence grants, exclusivity, non-compete, audit rights, and commercial covenants.

Categories were assigned to specialists by the author in consultation with the CUAD category descriptions; the assignment was fixed before any model was run. Each specialist's prompt is a focused reading instruction tuned to its category cluster, including category indicators and asymmetry guidance (e.g. liability caps require recall of monetary thresholds, renewal clauses require recall of conditional triggers). The specialist returns a verbatim-span answer in the same schema as the baselines. M1 thus combines two architectural commitments: routing and decomposition.

M6 — Consolidated specialist (ablation). M6 is the architectural ablation control. Its prompt is the textual union of the three specialist prompts of M1, concatenated into a single agent that handles every category itself, with no router, no orchestrator, and no per-category branching. The category indicator is selected based on the request, but no decomposition is performed. The point of M6 is methodological: it lets us decide, in Chapter 5, whether any benefit M1 enjoys over B1 is attributable to architectural decomposition or merely to the more elaborated prompt content that the specialists carry. Concretely, M1 vs. M6 isolates "architecture", and M6 vs. B1 isolates "prompt content".

4.4 Implementation Stack

Orchestration. All configurations are implemented as LangGraph `StateGraphs` [Lan24a], which let us express the orchestrator/router/specialist topology as typed nodes with declared transitions. Each node consumes and emits a structured run-state object containing the contract, the category, intermediate decisions (e.g. classifier verdict, routed specialist), and the in-progress answer. The graph topology is the executable representation of the architecture diagrams in Section 4.1.

Model client. A unified multi-provider model client wraps Anthropic, OpenAI, Google Vertex AI, and Ollama under one synchronous and one streaming interface. The client supports 19 models in total but only the 11 listed in Section 3.7 are used in official runs. Per call it records prompt tokens, completion tokens, cache-read tokens (where supported), wall-clock latency, and provider-billed cost. These diagnostics are written to the trace and aggregated downstream.

Prompt management. Prompts are stored in a YAML registry with a stable schema. The registry has two roots, `prompts/baselines/` (B1, B4) and `prompts/specialists/` (the three M1 specialists, the M6 consolidated prompt, the M0 classifier and extractor), and a separate `indicators/` directory holding the per-category indicator strings shared across configurations. The registry is loaded once per run and version-pinned in the run metadata.

Observability. Every model call is traced via Langfuse [Lan24b] with a structured trace format that records the configuration ID, the model identifier, the prompt template version, the input tokens, the output tokens, the parsed answer, and the routing or classification decision (where applicable). For M1, the trace records the orchestrator decision, the routed specialist, and the specialist's answer in a parent-child trace structure. We measured trace completeness (fraction of model calls for which all expected trace fields were populated) at 95.2% for M1 across the official runs; the remaining 4.8% are largely due to provider-side trace truncations on long contracts and are flagged in the run logs.

Data and persistence. The CUAD loader implements the 100,000-character filter, the natural-prior sampler, and the tier partition of Section 3.3. It is a deterministic component keyed by `seed=42`. Per-run outputs are persisted as JSONL files containing one record per (sample, configuration, model) prediction with full trace identifiers, allowing the statistical pipelines (Section 3.6) to be re-run from raw logs.

Language and tooling. The codebase is Python, managed with `uv`, and the statistical pipelines are built on `pandas` [The20], `numpy` [Har+20], and `scipy` [Vir+20]. Bootstrap resampling uses `numpy` random generators with run-level seeds for reproducibility.

4.5 Traceability and Auditability

A property of multi-stage agentic systems that is frequently advertised but rarely characterised is *auditability*: the claim that decomposition produces inspectable artefacts that

enable reviewer trust. We treat auditability as a system property of the implementation rather than as a hypothesis to be tested, and document here what the system records and what trade-offs that recording entails. This section addresses Sub-RQ3 of Chapter 1.

For each clause-level decision, the system captures: (i) the configuration ID and model identifier, (ii) the prompt template version and the rendered prompt, (iii) the model's full completion, (iv) the parsed final answer with the parser version, (v) the routing decision (M1) or classification verdict (M0) where applicable, and (vi) the per-call token, cost, and latency diagnostics. M1 traces are structured as parent-child relationships: the orchestrator's routing call is the parent, and the routed specialist's call is its child. M0 traces analogously expose the classifier as parent and the extractor (when invoked) as child.

This trace structure supports three concrete auditability properties. First, *decision attribution*: any final answer can be traced to the specific agent that produced it and the specific prompt version that elicited it. Second, *counterfactual replay*: because prompts and model identifiers are pinned per call, re-running the trace with a different model is mechanical, which lets a reviewer ask "would this contract have been flagged by a stronger model?" without re-running the full pipeline. Third, *rejection forensics*: when M0 silently returns "no related clause" on a category, the classifier's verdict is recorded explicitly, which makes the failure mode of Section 5.5.2 (over-rejection) directly visible at audit time.

The trade-offs are substantive. Verbose traces increase storage cost (the JSONL persistence layer is on the order of gigabytes per full sweep) and, when chains-of-thought are recorded, can themselves contain sensitive contract content; we therefore distinguish between trace fields that are safe to surface to operators (decisions, token counts) and those that should be access-controlled (full prompts and completions). Auditability is also a property of the *system*, not of the underlying model: the trace is faithful to the call graph but does not certify that the model's stated reasoning is its actual reasoning.

4.6 Experimental Infrastructure

Loader and sampling. The CUAD loader is the single source of truth for which contracts and which categories enter a run. It enforces the 100,000-character filter, applies the natural-prior sampler, and emits the tier label for every sample. For the official runs the loader produces approximately 2,999 samples per run and is identical across configurations, ensuring that pairing in Chapter 5's statistical pipeline is well-defined.

Statistical aggregation. A separate aggregation pipeline reads the per-run JSONL outputs, computes the per-sample correctness vectors, and emits the McNemar contingency tables, the paired bootstrap distributions, and the BH-corrected p-value tables. Aggregation is deterministic given the JSONL inputs and a fixed bootstrap seed. The pipeline emits machine-readable summaries that feed both the thesis tables and the dashboard.

Dashboard. A Next.js dashboard is provided for interactive exploration of experiment outputs, including per-model F_2 comparisons, per-tier breakdowns, per-call cost and latency views, and trace inspection by sample ID. The dashboard is not part of the statistical pipeline but is the primary interface used during error analysis and was the channel through which the over-extraction failure mode (Chapter 5) was first surfaced visually.

Test suite. An 81-test suite covers the metric implementations (F_1/F_2, Jaccard, laziness, over-extraction), the CUAD loader (filter, sampling determinism, tier assignment), the agent topologies (B1, B4, M0, M1, M6 each have at least one end-to-end test against a stub model), and the statistical pipelines (McNemar exact, paired bootstrap, BH correction, intersection logic for partial runs). The full suite passes prior to every official run. Notebook artefacts (`01_data_exploration.ipynb`, `02_workflow_test.ipynb`, `03_baseline_calibration.ipynb`, `04_multiagent_experiment.ipynb`) are retained as reproducible exploratory entry points.

Reproducibility. Reproducibility is supported by three commitments: deterministic sampling under `seed=42`; explicit version pinning of model identifiers in the run metadata; and persistence of the full trace bundle (prompts, completions, parsed answers, diagnostics) so that downstream statistics can be regenerated from raw logs without rerunning models.

5 Results

Note: the prose in this chapter is scaffolding. Quantitative claims carried over from the run logs are stated as such; all other numerical fields are placeholders to be filled once the final aggregation pipeline is re-run on the locked sample set.

5.1 Overview of Experimental Runs

This section establishes the coverage of the experimental sweep before any hypothesis is tested. We report the $5 \times 11 = 55$ official runs, the per-run sample count after natural-prior sampling, the total wall-clock, the total provider cost, and any partial-run intersections that affect downstream pairings.

Placeholder — Table 5.1: Coverage matrix. Rows: 11 models. Columns: 5 configurations (B1, B4, M0, M1, M6). Cells: per-run sample count, completion rate, and total provider cost. Bottom row: column totals.

Model	**B1**	**B4**	**M0**	**M1**	**M6**
Claude Haiku 4.5	–	–	–	–	–
Claude Sonnet 4	–	–	–	–	–
Claude Sonnet 4.6	–	–	–	–	–
Claude Opus 4.6	–	–	–	–	–
Gemini 2.5 Flash	–	–	–	–	–
Gemini 2.5 Pro	–	–	–	–	–
Gemini 3 Flash	–	–	–	–	–
Gemini 3.1 Pro	–	–	–	–	–
GPT-4.1	–	–	–	–	–
GPT-4.1 Mini	–	–	–	–	–
GPT-4.1 Nano	–	–	–	–	–
Total	–	–	–	–	–

Table 5.1: Coverage matrix: per-run sample count and completion rate for each (model, configuration) cell.

Placeholder — Table 5.2: Aggregate sample accounting. Total predictions analysed:

163,644 after dropping 1,297 ($\approx$ 0.79%) null-classification samples. Stratified by configuration and by tier.

Configuration	Total	Dropped	Analysed
B1	–	–	–
B4	–	–	–
M0	–	–	–
M1	–	–	–
M6	–	–	–
All	164,941	1,297	163,644

Table 5.2: Aggregate sample accounting after exclusion of malformed classification tokens.

Placeholder — Figure 5.1: Per-model wall-clock and cost stack. Per-model bar chart partitioned by configuration, supporting the cost-efficiency analysis of Section 5.5.3.

[Placeholder: per-model cost stack bar chart, partitioned by configuration (B1, B4, M0, M1, M6). To be generated from run-log aggregation.]

Figure 5.1: Per-model provider cost partitioned by configuration.

5.2 H1 — Multi-Agent vs. Single-Agent Baselines

This section presents the central H1 evidence: per-model McNemar exact tests on the family of pairwise configuration comparisons, paired bootstrap ΔF_2 effect sizes, and the BH-corrected p-value table. The headline finding is that no multi-agent configuration consistently outperforms the strongest single-agent baseline on F_2 across models: pooled mean F_2 across the eleven models lies within a 1.1-point band (B1 = 0.801, B4 = 0.796, M0 = 0.794, M1 = 0.790, M6 = 0.790). The verdict on H1 is therefore mixed/conditional rather than supportive in the pooled view, while a model-family interaction is documented (Claude-family models prefer B4, Gemini-family prefer B1, OpenAI-family prefer M0). M0 wins on 5 of 11 models against B1 and loses on 4 of 11.

Placeholder — Table 5.3: Per-model H1 results table. Rows: 11 models. Columns: F_2 for each of the 5 configurations, McNemar p for each multi-agent vs. best baseline, BH-corrected p, paired bootstrap ΔF_2 with 95% CI.

Placeholder — Figure 5.2: Architecture $\times$ model-family interaction heat map. Heat map of F_2 by (model family, configuration), summarising the family-conditional best architecture.

Model	F_2(B1)	F_2(B4)	F_2(M0)	F_2(M1)	F_2(M6)	p (best multi vs. $B^\star$)	p_{BH}
Claude Haiku 4.5	–	–	–	–	–	–	–
Claude Sonnet 4	–	–	–	–	–	–	–
Claude Sonnet 4.6	–	–	–	–	–	–	–
Claude Opus 4.6	–	–	–	–	–	–	–
Gemini 2.5 Flash	–	–	–	–	–	–	–
Gemini 2.5 Pro	–	–	–	–	–	–	–
Gemini 3 Flash	–	–	–	–	–	–	–
Gemini 3.1 Pro	–	–	–	–	–	–	–
GPT-4.1	–	–	–	–	–	–	–
GPT-4.1 Mini	–	–	–	–	–	–	–
GPT-4.1 Nano	–	–	–	–	–	–	–
Mean	0.801	0.796	0.794	0.790	0.790	—	

Table 5.3: H1 results: per-model F_2 by configuration, McNemar test, and BH-corrected p-value. Pooled means from run logs; per-model cells to be filled on final aggregation.

[Placeholder: 3 × 5 heat map of mean F_2 by (model family × configuration). Colour scale: low F_2 = white, high F_2 = TUMBlue. Best-in-family cell annotated with a border.]

Figure 5.2: Architecture × model-family interaction: mean F_2 by family and configuration.

Placeholder — Figure 5.3: Per-model ΔF_2 vs. B1 with 95% paired-bootstrap CIs, faceted by configuration.

[Placeholder: four-panel figure (one panel per non-B1 configuration) showing per-model ΔF_2 with 95% bootstrap CI. Positive = better than B1; zero line marked.]

Figure 5.3: Per-model ΔF_2 relative to B1, with 95% paired-bootstrap confidence intervals, faceted by configuration.

H1 verdict. The strict null — that no multi-agent configuration outperforms the best single-agent baseline on F_2 — is not rejected at the family level after BH correction. Per-model interactions with model family are statistically and practically meaningful, however, and shape the discussion in Chapter 6.

5.3 H2 — Rare-Category Gap

This section reports per-tier F_2 across all five configurations and the paired bootstrap on the rare–common gap difference relative to B1. The main finding is that multi-agent architectures do *not* narrow the rare–common gap and may widen it for some configurations and models.

Placeholder — Table 5.4: Per-tier F_2. Rows: 11 models × 5 configurations. Columns: F_2 at common, moderate, rare tier; gap = $F_2^{common} - F_2^{rare}$; bootstrap CI on the gap.

Model	**Config**	F_2^{com}	F_2^{mod}	F_2^{rare}	**Gap**	**95% CI (gap)**
[To be populated from final aggregation]						

Table 5.4: Per-tier F_2 and rare–common gap by model and configuration.

Placeholder — Figure 5.4: Gap-difference forest plot. Per-model paired bootstrap ΔGap *for each multi-agent configuration vs. B1, with* 95% *CIs, ordered by model.*

[Placeholder: forest plot with one row per (model, configuration) pair (3 multi-agent configs × 11 models = 33 rows). x-axis: ΔGap (negative = multi-agent narrows the gap). Zero line marked. CI bars from paired bootstrap.]

Figure 5.4: Paired bootstrap ΔGap (multi-agent vs. B1) with 95% CIs. Negative values indicate a narrower rare–common gap under the multi-agent configuration.

H2 verdict. Rejected in its directional form. The rare–common gap is not consistently reduced by any multi-agent configuration and is widened in identifiable model–architecture pairs, with the dominant driver being the M0 classifier's behaviour on rare categories (Section 5.5.2).

5.4 H3 — Architecture vs. Prompts

This section isolates the contribution of architectural decomposition from the contribution of prompt content via the M1 vs. M6 contrast, with M0 vs. M6 as a secondary contrast separating the classification gate from decomposition. Headline figures from the run logs: M1 routing accuracy is 100.000% on 10 of 11 models (GPT-4.1 Nano: 99.033%); M1 and M6 are statistically indistinguishable on F_2 at the pooled level (mean $\Delta F_2 = 0.000$); M6 produces cleaner spans on Jaccard.

Placeholder — Table 5.5: H3 contrasts. Rows: 11 models. Columns: M1 vs. M6 McNemar p and BH-corrected p, paired bootstrap ΔF_2 with CI; same for M0 vs. M6; Jaccard for M1, M6, B1.

Model	ΔF_2(M1 vs. M6)	p_{BH}	ΔF_2(M0 vs. M6)	p_{BH}
[To be populated from final aggregation]				

Table 5.5: H3 contrasts: M1 vs. M6 and M0 vs. M6, with BH-corrected p-values.

Placeholder — Figure 5.5: Routing accuracy by model. Bar chart of M1 router agreement with the fixed category-to-specialist mapping.

[Placeholder: horizontal bar chart, one bar per model showing M1 routing accuracy (%). All bars near 100%; GPT-4.1 Nano at 99.033% highlighted.]

Figure 5.5: M1 router accuracy: agreement with the fixed category-to-specialist mapping, per model.

H3 verdict. Not rejected. The architectural decomposition in M1 does not produce a statistically detectable improvement over the consolidated specialist prompt of M6 on F_2, suggesting that any benefit of the specialist-prompt content is attributable to the prompt text, not the routing topology.

5.5 Additional Findings

5.5.1 The Near-Perfect Routing Accuracy Critique

This subsection examines the apparently strong but methodologically suspicious finding that M1's LLM router agrees with the fixed category-to-specialist mapping at 100% on 10 of 11 models. The finding is framed as evidence *against* the diagnostic value of router accuracy: when the routing task collapses to a category-name classification with disjoint, well-named clusters, frontier LLMs solve it trivially, and router accuracy ceases to discriminate among architectures. A deterministic dictionary lookup would achieve the same routing outcome at zero LLM cost.

Placeholder (optional) — Figure 5.6: Router agreement on a held-out perturbation set. The same routing prompt with adversarially renamed specialist labels, showing whether router behaviour degrades under relabelling.

5.5.2 The Classification-Gate Rejection Cascade

This subsection documents M0's dominant failure mode: the binary classifier overrejects on certain models, causing the extractor to be bypassed and the system to silently emit "no related clause". On Gemini 2.5 Flash the wrong-rejection rate is 40.9% versus

[Placeholder (optional): bar chart comparing routing accuracy under original vs. relabelled specialist names, per model. If the router relies on category-name cues, accuracy should degrade under relabelling.]

Figure 5.6: Router accuracy under original and adversarially relabelled specialist names (optional perturbation analysis).

2.7% for B1; on Gemini 2.5 Pro it is 31.7% versus 1.5%. Across all 11 models, between 70% and 98% of M0's false negatives are attributable to `predicted_no_clause` classifier verdicts.

Placeholder — Table 5.6: M0 rejection cascade. Rows: 11 models. Columns: classifier wrong-rejection rate, B1 laziness rate (for comparison), M0 false-negative rate, share of false negatives with `predicted_no_clause` true.

Model	M0 wrong-rej.	B1 laziness	M0 FN rate	FN due to gate
Claude Haiku 4.5	–	–	–	–
Claude Sonnet 4	–	–	–	–
Claude Sonnet 4.6	–	–	–	–
Claude Opus 4.6	–	–	–	–
Gemini 2.5 Flash	40.9%	2.7%	–	–
Gemini 2.5 Pro	31.7%	1.5%	–	–
Gemini 3 Flash	–	–	–	–
Gemini 3.1 Pro	–	–	–	–
GPT-4.1	–	–	–	–
GPT-4.1 Mini	–	–	–	–
GPT-4.1 Nano	–	–	–	–

Table 5.6: M0 classification-gate rejection cascade: wrong-rejection rate, B1 laziness, M0 false-negative rate, and share of false negatives attributable to the gate. Gemini rows from run logs; others to be populated.

Placeholder — Figure 5.7: M0 vs. B1 laziness on rare-tier categories, illustrating the disproportionate cost of the rejection cascade on the rare slice.

[Placeholder: grouped bar chart with one group per model, bars for M0 wrong-rejection rate and B1 laziness rate, restricted to rare-tier categories. Gemini family expected to show the largest divergence.]

Figure 5.7: M0 classifier wrong-rejection rate vs. B1 laziness rate on rare-tier categories, per model.

5.5.3 Cost-Efficiency Analysis

Multi-agent configurations cost 24%–70% more than B1 per run with no aggregate F_2 benefit, placing B1 on the Pareto frontier in the (F_2, cost) plane for the eleven models studied. This is a deployment-relevant finding rather than a refutation of agentic architectures in principle: the cost premium may be justified in workflows where the per-agent traces add operational value (e.g. obligation-tracking systems that need to surface the reasoning path to a reviewer). The conditions under which the premium is justified are discussed in Chapter 6.

Placeholder — Figure 5.8: Pareto plot of F_2 versus per-sample provider cost for all 55 (configuration, model) cells, with the empirical Pareto frontier highlighted.

[Placeholder: scatter plot, x-axis = mean per-sample cost ($), y-axis = mean F_2. One point per (configuration, model) cell, colour-coded by configuration. Pareto-optimal points (no dominated cells) connected by a step function. B1 expected to dominate in the lower-cost region.]

Figure 5.8: F_2 vs. per-sample provider cost for all 55 (configuration, model) cells. The Pareto frontier (no cell simultaneously cheaper and better) is indicated.

6 Discussion

6.1 When Multi-Agent Helps: Evidence for Pipeline Decomposition

6.2 When Multi-Agent Hurts: The LLM-Router Problem

6.3 Architecture vs Prompts: The H3 Verdict

6.4 Practitioner Perspective: Gaps Between Benchmarks and Real-World CLM

6.5 Connecting to Prior Work

6.6 Limitations

7 Conclusion and Outlook

7.1 Summary of Contributions

7.2 Answers to Research Questions

7.3 Outlook and Future Work

List of Figures

5.1 Per-model provider cost partitioned by configuration. 31
5.2 Architecture × model-family interaction: mean F_2 by family and configuration. 32
5.3 Per-model ΔF_2 relative to B1, with 95% paired-bootstrap confidence intervals, faceted by configuration. 32
5.4 Paired bootstrap ΔGap (multi-agent vs. B1) with 95% CIs. Negative values indicate a narrower rare–common gap under the multi-agent configuration. 33
5.5 M1 router accuracy: agreement with the fixed category-to-specialist mapping, per model. 34
5.6 Router accuracy under original and adversarially relabelled specialist names (optional perturbation analysis). 35
5.7 M0 classifier wrong-rejection rate vs. B1 laziness rate on rare-tier categories, per model. 35
5.8 F_2 vs. per-sample provider cost for all 55 (configuration, model) cells. The Pareto frontier (no cell simultaneously cheaper and better) is indicated. 36

List of Tables

2.1 Representative multi-agent systems by topology, routing, and verification structure. 12

4.1 The five evaluated configurations across the three design axes. 24

5.1 Coverage matrix: per-run sample count and completion rate for each (model, configuration) cell. 30
5.2 Aggregate sample accounting after exclusion of malformed classification tokens. 31
5.3 H1 results: per-model F_2 by configuration, McNemar test, and BH-corrected p-value. Pooled means from run logs; per-model cells to be filled on final aggregation. 32
5.4 Per-tier F_2 and rare–common gap by model and configuration. 33
5.5 H3 contrasts: M1 vs. M6 and M0 vs. M6, with BH-corrected p-values. . 34
5.6 M0 classification-gate rejection cascade: wrong-rejection rate, B1 laziness, M0 false-negative rate, and share of false negatives attributable to the gate. Gemini rows from run logs; others to be populated. 35

Bibliography

[BH95] Y. Benjamini and Y. Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." In: *Journal of the Royal Statistical Society, Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.

[Bog+25] T. Bogavelli et al. *AgentArch: A Comprehensive Benchmark to Evaluate Agent Architectures in Enterprise*. ServiceNow Research. Full author list at arXiv:2509.10769. 2025. arXiv: 2509.10769.

[Car+20] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky. "With Little Power Comes Great Responsibility." In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*. 2020, pp. 9263–9274. DOI: 10.18653/v1/2020.emnlp-main.745. arXiv: 2010.06595.

[Cem+25] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. *Why Do Multi-Agent LLM Systems Fail?* 2025. arXiv: 2503.13657.

[Cha+20] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos. "LEGAL-BERT: The Muppets Straight Out of Law School." In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020, pp. 2898–2904. arXiv: 2010.02559.

[Cha+24] A. Chatterjee, H. S. V. N. S. K. Renduchintala, S. Bhatia, and T. Chakraborty. "POSIX: A Prompt Sensitivity Index for Large Language Models." In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024, pp. 14550–14565. arXiv: 2410.02185.

[Cui+24] J. Cui, M. Ning, Z. Li, B. Chen, Y. Yan, H. Li, B. Ling, Y. Tian, and L. Yuan. *ChatLaw: A Multi-Agent Collaborative Legal Assistant with Knowledge Graph Enhanced Mixture-of-Experts Large Language Model*. Version 2 (2024); v1 (2023) is the original retrieval-augmented system. 2024. arXiv: 2306.16092.

[Dev+19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*. 2019, pp. 4171–4186. arXiv: 1810.04805.

[Dro+18] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing." In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*. 2018, pp. 1383–1392.

[EC16] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4.5.2016, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj. 2016.

[EC24] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L 2024/1689, 12.7.2024. http://data.europa.eu/eli/reg/2024/1689/oj. 2024.

[Efr79] B. Efron. "Bootstrap Methods: Another Look at the Jackknife." In: *The Annals of Statistics* 7.1 (1979), pp. 1–26. DOI: 10.1214/aos/1176344552.

[Gao+25] M. Gao, Y. Li, B. Liu, Y. Yu, P. Wang, C.-Y. Lin, and F. Lai. *Single-agent or Multi-agent Systems? Why Not Both?* 2025. arXiv: 2505.18286.

[Har+20] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. "Array Programming with NumPy." In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[Hat+23] J. Hatzius, J. Briggs, D. Kodnani, and G. Pierdomenico. *The Potentially Large Effects of Artificial Intelligence on Economic Growth (Briggs/Kodnani)*. Goldman Sachs Global Economics Analyst. Published 26 March 2023. Primary report is paywalled; widely cited in *The Globe and Mail* and *Law.com*. Mar. 2023.

[Hen+21] D. Hendrycks, C. Burns, A. Chen, and S. Ball. "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review." In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS 2021)*. 2021. arXiv: 2103.06268.

[Hon+24] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework." In: *12th International Conference on Learning Representations (ICLR 2024)*. Oral presentation. 2024. arXiv: 2308.00352.

[KM21] Y. Koreeda and C. D. Manning. "ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts." In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 1907–1919. arXiv: 2110.01799.

[Lam+16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer. "Neural Architectures for Named Entity Recognition." In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2016)*. 2016, pp. 260–270. arXiv: 1603.01360.

[Lan24a] LangChain Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. Software library. https://github.com/langchain-ai/langgraph. 2024.

[Lan24b] Langfuse GmbH. *Langfuse: Open-Source LLM Engineering Platform*. Software / documentation. https://langfuse.com/ ; https://github.com/langfuse/langfuse. 2024.

[Li+23] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. "CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.17760.

[Liu+25] S. Liu, Z. Li, R. Ma, H. Zhao, and M. Du. *ContractEval: Benchmarking LLMs for Clause-Level Legal Risk Identification in Commercial Contracts*. 2025. arXiv: 2508.03080.

[LMP01] J. D. Lafferty, A. McCallum, and F. C. N. Pereira. "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data." In: *Proceedings of the 18th International Conference on Machine Learning (ICML 2001)*. 2001, pp. 282–289.

[Mar+24] L. Martin, N. Whitehouse, S. Yiu, L. Catterson, and R. Perera. *Better Call GPT: Comparing Large Language Models Against Lawyers*. Onit AI Centre of Excellence. Vendor-funded; directional findings replicated by independent evaluations. 2024. arXiv: 2401.16212.

[McN47] Q. McNemar. "Note on the Sampling Error of the Difference between Correlated Proportions or Percentages." In: *Psychometrika* 12.2 (1947), pp. 153–157. DOI: 10.1007/BF02295996.

[Par+23] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. "Generative Agents: Interactive Simulacra of Human Behavior." In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*. 2023. DOI: 10.1145/3586183.3606763. arXiv: 2304.03442.

[Rap+25] P. Raptopoulos, G. Filandrianos, M. Lymperaiou, and G. Stamou. "PAKTON: A Multi-Agent Framework for Question Answering in Long Legal Agreements." In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. Oral presentation. 2025. arXiv: 2506.00608.

[RM05] S. Riezler and J. T. Maxwell III. "On Some Pitfalls in Automatic Evaluation and Significance Testing for MT." In: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 2005, pp. 57–64.

[San25] M. Sanwal. *Layered Chain-of-Thought Prompting for Multi-Agent LLM Systems: A Comprehensive Approach to Explainable Large Language Models*. 2025. arXiv: 2501.18645.

[Shi+23] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. "Reflexion: Language Agents with Verbal Reinforcement Learning." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.11366.

[The20] The pandas development team. *pandas-dev/pandas: Pandas*. Zenodo. Cite the Zenodo release matching the version used in the study. 2020. DOI: 10.5281/zenodo.3509134.

[Tug+20] D. Tuggener, P. von Däniken, T. Peetz, and M. Cieliebak. "LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts." In: *Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020)*. 2020, pp. 1235–1241.

[Vir+20] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[Wan+23a] S. H. Wang, A. Scardigli, L. Tang, W. Chen, D. Levkin, A. Chen, S. Ball, T. Woodside, O. Zhang, and D. Hendrycks. "MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding." In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. 2023, pp. 16369–16382. arXiv: 2301.00876.

[Wan+23b] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." In: *11th International Conference on Learning Representations (ICLR 2023)*. 2023. arXiv: 2203.11171.

[Wan+24] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou. *Mixture-of-Agents Enhances Large Language Model Capabilities*. Also published at ICLR 2025. 2024. arXiv: 2406.04692.

[Wei+22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." In: *Advances in Neural Information Processing Systems (NeurIPS 2022)*. 2022. arXiv: 2201.11903.

[Wor25] World Commerce & Contracting. *Contract Management: An Overlooked Driver of Business Agility and Financial Performance*. WorldCC Whitepaper. Available at https://www.worldcc.com/Portals/IACCM/Reports/Contract%20Management%20Whitepaper.pdf. 2025.

[Wu+23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework*. Also published at COLM 2024. 2023. arXiv: 2308.08155.

[WY25] Z. Wang and B. L. Yuan. *L-MARS: Legal Multi-Agent Workflow with Orchestrated Reasoning and Agentic Search*. 2025. arXiv: 2509.00761.

[Yao+23a] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2305.10601.

[Yao+23b] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." In: *11th International Conference on Learning Representations (ICLR 2023)*. 2023. arXiv: 2210.03629.